Connect 4 Intelligent Game Agent

Minimax Algorithm with Alpha-Beta Pruning

Artificial Intelligence in Action
Applied Programming Project

Student: **Ipek**

Course: Artificial Intelligence

Project Topic: Intelligent Game Agent

1. Introduction

This report documents the design, implementation and evaluation of an intelligent game-playing agent for the classic two-player game **Connect 4**. The agent is built around the **Minimax algorithm**, optimised with **Alpha-Beta Pruning**, and plays on a standard 6 x 7 board against a human opponent. The project satisfies the requirements of the *Intelligent Game Agent* track by producing a fully working application, a graphical user interface for live play, and a quantitative comparison between plain Minimax and its Alpha-Beta optimised counterpart.

Connect 4 was chosen over simpler games such as Tic-Tac-Toe because its game tree is substantially larger (the game has roughly 4.5 trillion legal positions), which makes the practical benefit of Alpha-Beta Pruning clearly observable. This allows the report to demonstrate, with concrete numbers, how a classical optimisation technique transforms a naively intractable search into one that runs comfortably in real time.

2. Problem Definition

Connect 4 is a two-player, zero-sum, perfect-information, deterministic game. The board consists of 6 rows and 7 columns. Players take turns dropping a coloured disc into any non-full column, and the disc falls to the lowest empty cell of that column due to gravity. The first player to align four of their own discs horizontally, vertically or diagonally wins. If the board fills without such a line, the game is a draw.

From an AI perspective the problem is formalised as a search over a game tree whose branching factor is at most 7 (the number of columns) and whose maximum depth is 42 (the total number of cells). Although the game is known to be solved (Allis, 1988), producing an optimal agent at every depth is unnecessary for this project; instead we build a depth-limited agent whose difficulty can be tuned via the search depth parameter.

3. Theoretical Background

3.1 The Minimax Algorithm

Minimax is a recursive decision procedure for two-player zero-sum games. The two players are labelled **MAX** (the agent, whose score we try to maximise) and **MIN** (the opponent, assumed to play optimally against MAX). At every node of the game tree, MAX chooses the child with the highest value, and MIN chooses the child with the lowest value. Formally, for a node n:

$$
\begin{aligned}
&\textbf{minimax(n)} = \\
&\quad \textbf{utility(n)} && \text{if n is terminal} \\
&\quad \textbf{max}_{s \in \text{children(n)}}\ \text{minimax(s)} && \text{if n is a MAX node} \\
&\quad \textbf{min}_{s \in \text{children(n)}}\ \text{minimax(s)} && \text{if n is a MIN node}
\end{aligned}
$$

Because the full game tree is too large to explore exhaustively, in practice Minimax is run to a fixed **search depth d**. Nodes at depth d that are not terminal are scored with a **heuristic evaluation function** that estimates how favourable the position is for MAX.

The time complexity of plain Minimax is $\mathbf{O(b^d)}$ where b is the branching factor and d is the search depth. For Connect 4, $b \leq 7$, so depth 6 already explores on the order of $7^6 \approx 117\,000$ leaf evaluations. Deeper searches quickly become impractical without optimisation.

3.2 Alpha-Beta Pruning

Alpha-Beta Pruning is an optimisation of Minimax that produces the **exact same result** while exploring far fewer nodes. It carries two bounds through the recursion:

• α (alpha): the best value MAX can already guarantee along the path to the root.
• β (beta): the best value MIN can already guarantee along the path to the root.

Whenever $\alpha \geq \beta$ at a node, the remaining children of that node cannot affect the final decision because a rational opponent would never allow play to reach them. The recursion can therefore **prune** them, skipping potentially huge sub-trees.

In the best case (when children are ordered from best to worst at every node) the complexity drops to $\mathbf{O(b^{d/2})}$, which is equivalent to doubling the effective search depth for the same budget. Our implementation exploits this by ordering candidate moves from the centre column outwards, which is a well-known rule of thumb in Connect 4 because the centre column participates in the greatest number of possible four-in-a-row lines.

3.3 Heuristic Evaluation Function

Because Connect 4 cannot be searched to terminal nodes at reasonable depths, we define a heuristic that scores any non-terminal position from the perspective of the AI. Our heuristic combines two ideas:

(i) Centre control bonus. Every piece the AI has in the centre column contributes a small positive score. The centre is strategically strong because more potential four-in-a-row lines pass through it than any other column.

(ii) Window scoring. The board is scanned with a sliding 4-cell window in all four directions (horizontal, vertical, and both diagonals). Each window is scored according to its composition:

Window contents	Score	Rationale
4 AI pieces	+100	Already winning
3 AI + 1 empty	+10	One move from winning
2 AI + 2 empty	+5	Developing threat
3 opponent + 1 empty	-8	Must be blocked
2 opponent + 2 empty	-2	Mild defensive concern
Mixed pieces	0	Window is blocked

Table 1. Heuristic window scoring scheme.

The asymmetry between the +10 offensive bonus and the -8 defensive penalty encourages the agent to play offensively when its threats are as strong as the opponent's, rather than blocking reactively. Terminal wins and losses are scored with much larger magnitudes (±100 000) so that they always dominate heuristic values.

4. Code Architecture

The project is organised into a small number of focused Python modules, each with a single responsibility. This modular design keeps the game logic cleanly separated from the search algorithm and the user interface, and makes it straightforward to benchmark or replace any component.

Module	Responsibility
`board.py`	Game state, move application, win detection, NumPy representation.
`evaluator.py`	Heuristic position evaluation (centre bonus + window scoring).
`ai_agent.py`	Minimax and Alpha-Beta search, difficulty levels, performance statistics.
`gui.py`	Pygame interface: menu, board rendering, AI-statistics side panel.
`terminal_game.py`	Text-mode Human-vs-AI and AI-vs-AI demonstration modes.
`benchmark.py`	Systematic comparison of Minimax vs Alpha-Beta across depths.
`generate_plots.py`	Produces the figures used in this report.
`main.py`	Top-level launcher menu.

Table 2. Module responsibilities.

4.1 Core Search Implementation

Both Minimax and Alpha-Beta are implemented as pure recursive functions operating on a shared `Connect4Board` object. A `SearchStats` data class is passed through the recursion so that every call can be counted, which is essential for the empirical comparison in Section 5. The core pseudocode for the Alpha-Beta routine is:

```
function alphabeta(board, depth, α, β, maximizing):
  if terminal(board) or depth = 0:
    return evaluate(board)
  if maximizing:
    value ← -∞
    for each move in order_moves(board):
      value ← max(value, alphabeta(child, depth-1, α, β, False))
      α ← max(α, value)
      if α ≥ β: break    // prune
    return value
  else:
    value ← +∞
    for each move in order_moves(board):
      value ← min(value, alphabeta(child, depth-1, α, β, True))
      β ← min(β, value)
      if α ≥ β: break    // prune
    return value
```

4.2 Difficulty Levels

The user can choose between three difficulty levels on the main menu. Each level corresponds to a different search depth:

Level	Search depth	Typical response time	Playing strength
Easy	2	< 0.05 s	Blocks immediate threats only

Medium	4	0.2 - 1.0 s	Plans a few moves ahead
Hard	6	1 - 5 s	Rarely makes tactical mistakes

Table 3. Difficulty levels.

5. Experimental Results

To quantify the benefit of Alpha-Beta Pruning, both algorithms were run on identical board positions with search depths ranging from 1 to 5. Three different starting positions were used (an empty board, a position after 4 moves, and a position after 8 moves) to check that the speedup is consistent across game phases. All measurements were taken on the same machine with no concurrent load.

5.1 Search Tree Size

The number of recursive calls (nodes visited) grows exponentially with depth for plain Minimax, but far more slowly for Alpha-Beta. Figure 1 plots the two curves on a logarithmic scale.

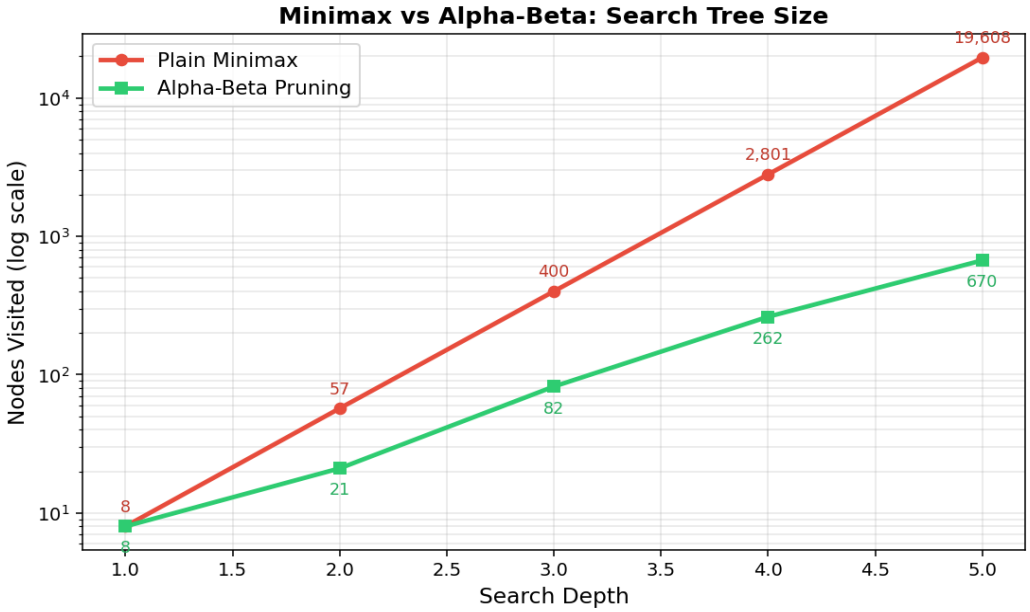

Figure 1. Nodes visited by Minimax and Alpha-Beta as a function of search depth, measured on the empty board.

5.2 Computation Time

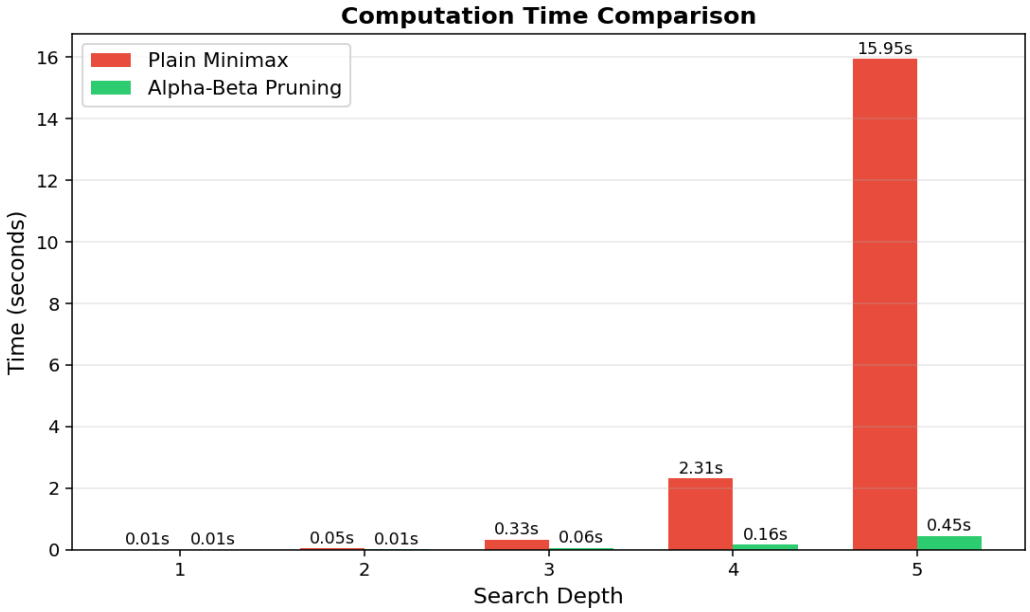

Figure 2. Wall-clock time required to compute the best move.

5.3 Speedup Factor

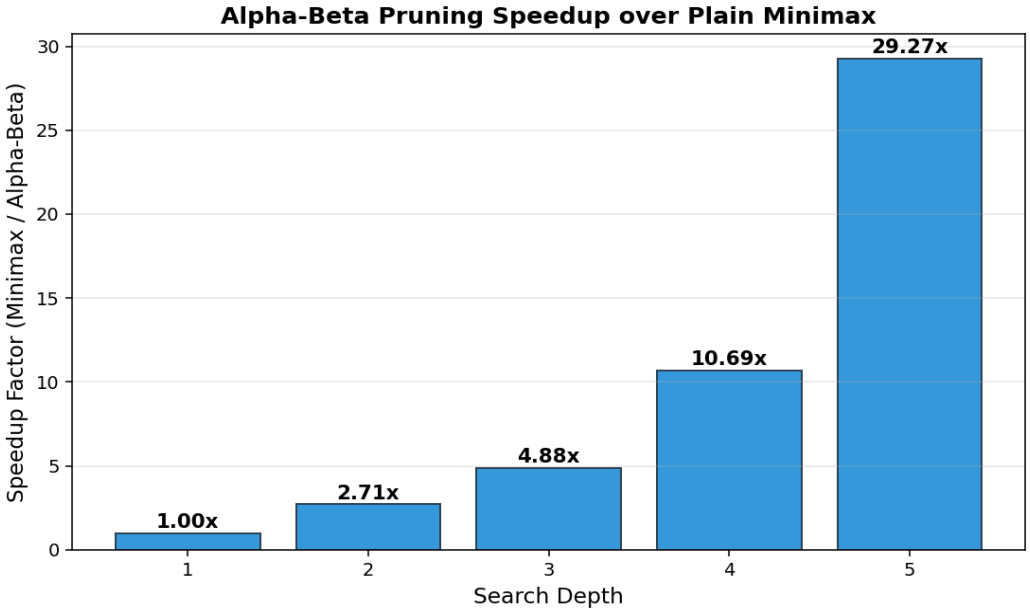

Figure 3. Ratio of nodes visited (Minimax / Alpha-Beta). The speedup grows with depth, reaching ~29x at depth 5.

5.4 Combined Results Table

Depth	Minimax nodes	Alpha-Beta nodes	Speedup	Minimax time	Alpha-Beta time
1	8	8	1.00x	0.008 s	0.007 s
2	57	21	2.71x	0.063 s	0.014 s
3	400	82	4.88x	0.375 s	0.064 s
4	2,801	262	10.69x	2.673 s	0.219 s
5	19,608	670	29.27x	17.720 s	0.561 s

Table 4. Empty-board benchmark results. The speedup grows super-linearly with depth, confirming the O(b^(d/2)) complexity advantage predicted by theory.

A crucial sanity check was also performed: for every depth and position tested, plain Minimax and Alpha-Beta returned the **same best move**. This confirms that pruning accelerates the search without altering the decision, which is the defining property of Alpha-Beta.

6. Application Demonstration

The main deliverable is a Pygame-based graphical application. The start screen (Figure 4) lets the user choose a difficulty level; the game screen (Figure 5) displays the board, allows the human to play by clicking a column, and shows live AI statistics on the right-hand panel (nodes visited, branches pruned, think time, and the current heuristic score).

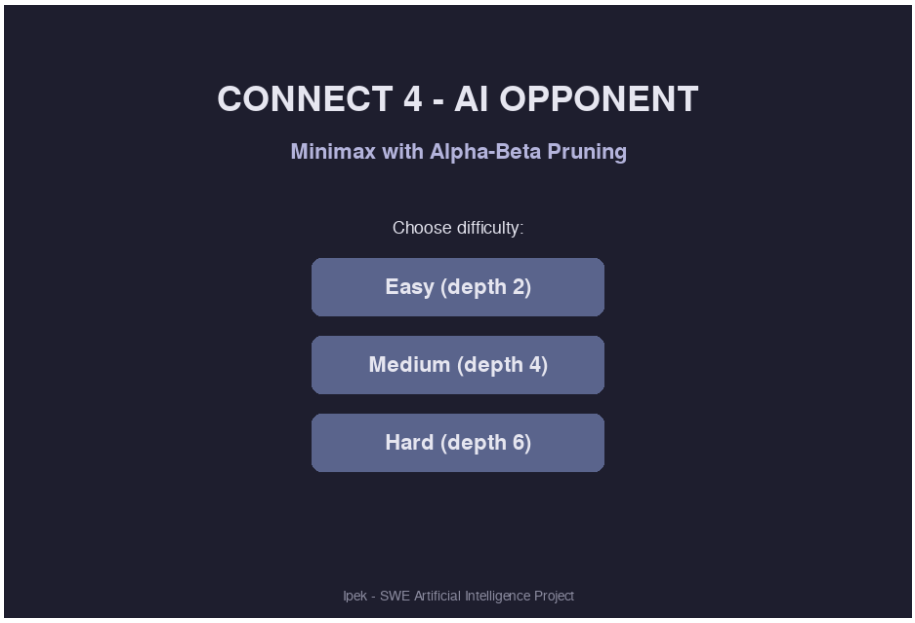

Figure 4. Start menu with difficulty selection.

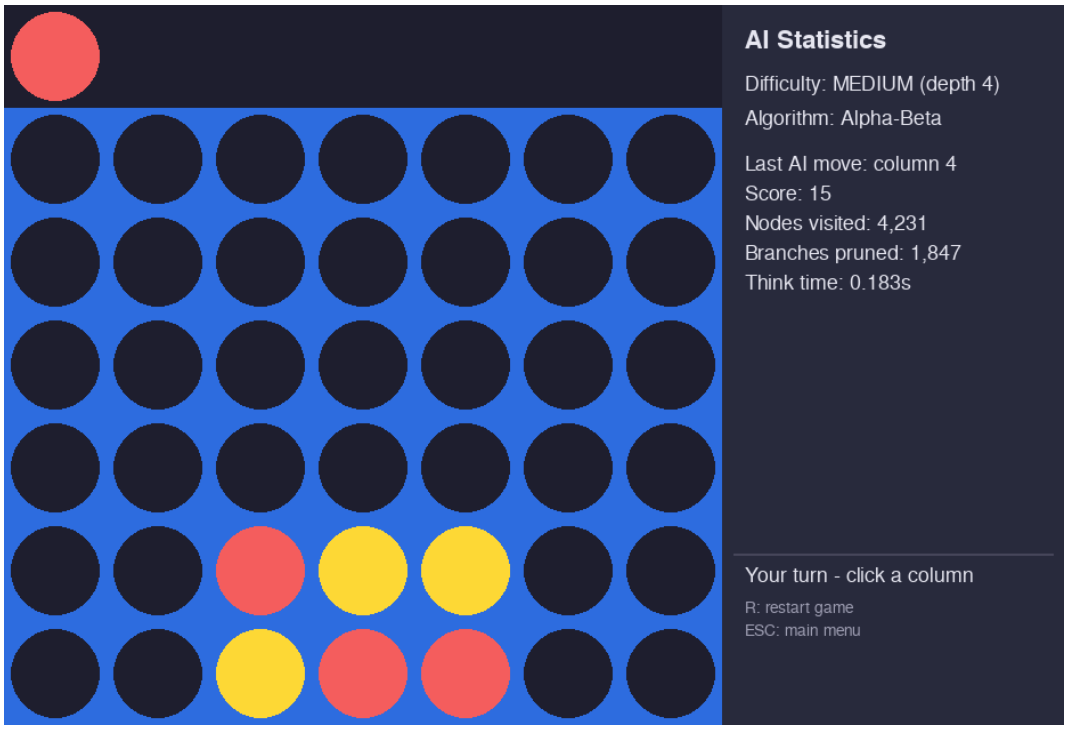

Figure 5. In-game view. The right panel reports that the last AI move visited 4,231 nodes with 1,847 branches pruned in 0.18 s.

An alternative terminal-mode interface is also provided, together with an AI-vs-AI mode in which two agents at different search depths play each other. In testing, a depth-5 agent defeated a depth-2 agent in nearly every match, providing a simple qualitative check that deeper search yields stronger play.

7. Conclusion

This project produced a complete, working Connect 4 agent that implements both Minimax and Minimax with Alpha-Beta Pruning. The agent is playable through a polished graphical interface with selectable difficulty and live search statistics, and its behaviour has been verified on critical tactical positions (it correctly blocks opponent wins and executes winning threats). The experimental comparison confirms the theoretical advantage of Alpha-Beta: at search depth 5 it visits roughly 29 times fewer nodes than plain Minimax while returning the identical move, reducing the per-move computation time from 17.7 seconds to 0.56 seconds - the difference between an unusable and a responsive agent.

Possible extensions include iterative deepening with time control, a transposition table to cache repeated positions, further move-ordering heuristics (such as killer moves), and a learned evaluation function trained on self-play data. Each of these would push the effective search depth higher without sacrificing responsiveness.

8. References

[1] Russell, S. & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson. Chapters 5-6 (Adversarial Search).

[2] Allis, L. V. (1988). *A Knowledge-Based Approach of Connect-Four: The Game is Solved, White Wins.* Master's Thesis, Vrije Universiteit Amsterdam.

[3] Knuth, D. E. & Moore, R. W. (1975). *An Analysis of Alpha-Beta Pruning.* Artificial Intelligence, 6(4), 293-326.

[4] Shannon, C. E. (1950). *Programming a Computer for Playing Chess.* Philosophical Magazine, 41(314), 256-275.

[5] Pygame Community. *Pygame Documentation.* https://www.pygame.org/docs/

[6] NumPy Developers. *NumPy User Guide.* https://numpy.org/doc/